

Cloud Data Loss Prevention 總覽

來源：<https://codelabs.developers.google.com/codelabs/cloud-dlp-overview?hl=zh-tw>

步驟數：7

在這個程式碼研究室中，系統會透過指令列介面向使用者介紹 DLP API。使用者必須下載專案程式碼，並檢查範例目錄中的一些工具及其基礎函式。

目錄

1. [1. 總覽](#)
2. [2. 開始設定](#)
3. [3. 檢查字串和檔案](#)
4. [4. 去識別化](#)
5. [5. 遮蓋字串和圖片](#)
6. [6. 清除所用資源](#)
7. [7. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽

Cloud Data Loss Prevention (DLP) 是一項全代管服務，可協助您探索、分類及保護機密資訊。本程式碼研究室將介紹 Cloud DLP API 的基本功能，並示範如何使用各種 API 功能來保護資料。

執行步驟

- 使用 DLP 檢查字串和檔案中是否有相符的資訊類型
- 瞭解去識別化技巧並使用 DLP 將資料去識別化
- 瞭解如何使用格式保留加密 (FPE) 重新識別已去識別化的資料
- 使用 DLP 遮蓋字串和圖片中的資訊類型

軟硬體需求

- 已設定帳單的 Google Cloud 專案。如果沒有，請[建立一個](#)。

注意：本程式碼研究室部署的解決方案會產生計費活動。如要瞭解 Google Compute Engine 定價，請參閱[這篇文章](#)。Google Cloud Platform 新使用者享有[價值 \\$300 美元的免費試用期](#)。

步驟 2 · 約 2 分鐘

2. 開始設定

本程式碼研究室完全可在 Google Cloud Platform 上執行，不需進行任何本機安裝或設定。

Cloud Shell

在本程式碼研究室中，我們將使用 [Cloud Shell](#)，透過指令列佈建及管理不同的雲端資源和服務。

下載隨附專案存放區：

```
git clone https://github.com/googleapis/nodejs-dlp
```

下載專案程式碼後，請切換至 `samples` 目錄，然後安裝必要的 Node.js 套件：

```
cd samples && npm install
```

請執行下列 `gcloud` 指令，設定正確的專案：

```
gcloud config set project [PROJECT_ID]
```

啟用 API

我們需要在專案中啟用下列 API：

- **Cloud Data Loss Prevention API** - 提供多種功能，可偵測文字、圖像和 Google Cloud Platform 儲存空間存放區中的隱私敏感內容，並進行風險分析和去識別化
- **Cloud Key Management Service (KMS) API** - Google Cloud KMS 可讓客戶管理加密金鑰，並使用這些金鑰執行加密編譯作業。

執行下列 `gcloud` 指令，啟用必要的 API：

```
gcloud services enable dlp.googleapis.com cloudkms.googleapis.com \
--project ${GOOGLE_CLOUD_PROJECT}
```

步驟 3 · 約 4 分鐘

3. 檢查字串和檔案

前一個步驟下載的專案樣本目錄中，有多個 JavaScript 檔案，這些檔案能執行 Cloud DLP 的不同功能。`inspect.js` 會檢查提供的字串或檔案是否含有機密資訊類型。

如要測試這項功能，可以提供 `string` 選項和含有潛在機密資訊的字串範例：

```
node inspect.js -c $GOOGLE_CLOUD_PROJECT \  
string 'My email address is jenny@somedomain.com and you can call me at 555-867-5309'
```

輸出內容會顯示每個相符資訊類型的發現項目，包括：

引號：範本會指定

InfoType：從該字串偵測到的資訊類型。如需可能出現的資訊類型完整清單，請參閱[這篇文章](#)。根據預設，`inspect.js` 只會檢查 `CREDIT_CARD_NUMBER`、`PHONE_NUMBER` 和 `EMAIL_ADDRESS` 資訊類型

Likelihood：系統會根據各結果代表相符項目的可能程度分類；範圍從 `VERY_UNLIKELY` 到 `VERY_LIKELY`。

上述指令要求的結果如下：

```
Findings:  
  Quote: jenny@somedomain.com  
  Info type: EMAIL_ADDRESS  
  Likelihood: LIKELY  
  Quote: 555-867-5309  
  Info type: PHONE_NUMBER  
  Likelihood: VERY_LIKELY
```

同樣地，我們也可以檢查檔案中的資訊類型。查看範例 `accounts.txt` 檔案：

[resources/accounts.txt](#)

```
My credit card number is 1234 5678 9012 3456, and my CVV is 789.
```

再次執行 `inspect.js`，這次使用檔案選項：

```
node inspect.js -c $GOOGLE_CLOUD_PROJECT file resources/accounts.txt
```

結果：

```
Findings:  
  Quote: 5678 9012 3456  
  Info type: CREDIT_CARD_NUMBER  
  Likelihood: VERY_LIKELY
```

無論是哪種查詢，我們都可以依據可能性或資訊類型限制結果。例如：

```
node inspect.js -c $GOOGLE_CLOUD_PROJECT \  
string 'Call 900-649-2568 or email me at anthony@somedomain.com' \  
-m VERY_LIKELY
```

指定 `VERY_LIKELY` 做為最低可能性後，系統會排除任何低於 `VERY_LIKELY` 的相符項目：

Findings:

```
Quote: 900-649-2568  
Info type: PHONE_NUMBER  
Likelihood: VERY_LIKELY
```

不受限制的完整結果如下：

Findings:

```
Quote: 900-649-2568  
Info type: PHONE_NUMBER  
Likelihood: VERY_LIKELY  
Quote: anthony@somedomain.com  
Info type: EMAIL_ADDRESS  
Likelihood: LIKELY
```

同樣地，我們可以指定要檢查的資訊類型：

```
node inspect.js -c $GOOGLE_CLOUD_PROJECT \  
string 'Call 900-649-2568 or email me at anthony@somedomain.com' \  
-t EMAIL_ADDRESS
```

如果找到指定資訊類型，系統只會傳回該類型：

Findings:

```
Quote: anthony@somedomain.com  
Info type: EMAIL_ADDRESS  
Likelihood: LIKELY
```

以下非同步函式會使用 API 檢查輸入內容：

inspect.js

```
async function inspectString(  
  callingProjectId,  
  string,  
  minLikelihood,  
  maxFindings,  
  infoTypes,  
  customInfoTypes,  
  includeQuote  
) {  
  ...  
}
```

系統會使用上述參數的引數建構要求物件，再將該要求提供給 `inspectContent` 函式，以取得回應並產生輸出內容：

inspect.js

```
// Construct item to inspect  
const item = {value: string};  
  
// Construct request  
const request = {  
  parent: dlp.projectPath(callingProjectId),  
  inspectConfig: {  
    infoTypes: infoTypes,  
    customInfoTypes: customInfoTypes,  
    minLikelihood: minLikelihood,  
    includeQuote: includeQuote,  
    limits: {  
      maxFindingsPerRequest: maxFindings,  
    },  
  },  
  item: item,  
};  
...  
...  
const [response] = await dlp.inspectContent(request);
```

4. 去識別化

除了檢查及偵測機密資料，Cloud DLP 還能執行去識別化作業。去識別化是從資料中移除識別資訊的程序。API 會偵測資訊類型定義的機密資料，然後使用去識別化轉換來遮蔽、刪除或隱藏資料。

`deid.js` 將以多種方式示範去識別化。最簡單的去識別化方法是使用遮罩：

```
node deid.js deidMask -c $GOOGLE_CLOUD_PROJECT \  
"My order number is F12312399. Email me at anthony@somedomain.com"
```

API 會遮蓋相符資訊類型的字元，並替換為其他字元，預設為「*」。輸出內容如下：

```
My order number is F12312399. Email me at *****
```

您會發現字串中的電子郵件地址已經過模糊化處理，任意訂單編號則維持不變。(您也能自訂資訊類型，但這不在本程式碼研究室的範圍)。

以下是使用 DLP API 進行遮蓋去識別化的函式：

`deid.js`

```
async function deidentifyWithMask(  
  callingProjectId,  
  string,  
  maskingCharacter,  
  numberToMask  
) {  
  ...  
}
```

這些引數會再次用於建構要求物件，但這次要求物件會提供給 `deidentifyContent` 函式：

`deid.js`


```
// Construct deidentification request
const item = {value: string};
const request = {
  parent: dlp.projectPath(callingProjectId),
  deidentifyConfig: {
    infoTypeTransformations: {
      transformations: [
        {
          primitiveTransformation: {
            characterMaskConfig: {
              maskingCharacter: maskingCharacter,
              numberToMask: numberToMask,
            },
          },
        },
      ],
    },
  },
  item: item,
};
...
const [response] = await dlp.deidentifyContent(request);
```

使用格式保留加密去識別化

DLP API 也提供使用加密編譯金鑰加密機密資料值的功能。

首先，我們將使用 [Cloud KMS](#) 建立金鑰環：

```
gcloud kms keyrings create dlp-keyring --location global
```

現在我們可以建立用來加密資料的金鑰：

```
gcloud kms keys create dlp-key \
  --purpose='encryption' \
  --location=global \
  --keyring=dlp-keyring
```

DLP API 會接受以我們建立的 KMS 金鑰加密的包裝金鑰。我們可以產生要包裝的隨機字串。稍後我們需要這項資訊，才能重新識別：

```
export AES_KEY=`head -c16 < /dev/random | base64 -w 0`
```

現在可以使用 KMS 金鑰加密字串。這會產生二進位檔案，其中包含加密字串做為密文：

```
echo -n $AES_KEY | gcloud kms encrypt \
--location global \
--keyring dlp-keyring \
--key dlp-key \
--plaintext-file - \
--ciphertext-file ./ciphertext.bin
```

現在可以使用 `deid.js`，透過加密方式將下列範例字串中的電話號碼去識別化：

```
node deid.js deidFpe -c $GOOGLE_CLOUD_PROJECT \
"My client's cell is 9006492568" `base64 -w 0 ciphertext.bin` \
projects/${GOOGLE_CLOUD_PROJECT}/locations/global/keyRings/dlp-
keyring/cryptoKeys/dlp-key \
-s PHONE_NUMBER
```

輸出內容會傳回字串，其中相符的資訊類型會替換為加密字串，並以 `-s` 旗標表示的資訊類型為前置字元：

```
My client's cell is PHONE_NUMBER(10):vSt55z79nR
```

我們來看看用於去識別字串的函式：

deid.js

```
async function deidentifyWithFpe(
  callingProjectId,
  string,
  alphabet,
  surrogateType,
  keyName,
  wrappedKey
) {
  ...
}
```

這些引數會用於建構 `cryptoReplaceFfxFpeConfig` 物件：

deid.js

```
const cryptoReplaceFfxFpeConfig = {
  cryptoKey: {
    kmsWrapped: {
      wrappedKey: wrappedKey,
      cryptoKeyName: keyName,
    },
  },
  commonAlphabet: alphabet,
};
if (surrogateType) {
  cryptoReplaceFfxFpeConfig.surrogateInfoType = {
    name: surrogateType,
  };
}
```

接著，透過 `deidentifyContent` 函式，在 API 的要求中使用 `cryptoReplaceFfxFpeConfig` 物件：

deid.js

```
// Construct deidentification request
const item = {value: string};
const request = {
  parent: dlp.projectPath(callingProjectId),
  deidentifyConfig: {
    infoTypeTransformations: {
      transformations: [
        {
          primitiveTransformation: {
            cryptoReplaceFfxFpeConfig: cryptoReplaceFfxFpeConfig,
          },
        },
      ],
    },
  },
  item: item,
};

try {
  // Run deidentification request
  const [response] = await dlp.deidentifyContent(request);
```

重新識別資料

如要重新識別資料，DLP API 會使用上一步建立的密文：

```
node deid.js reidFpe -c $GOOGLE_CLOUD_PROJECT \
"<YOUR_DEID_OUTPUT>" \
PHONE_NUMBER `base64 -w 0 ciphertext.bin` \
projects/${GOOGLE_CLOUD_PROJECT}/locations/global/keyRings/dlp-
keyring/cryptoKeys/dlp-key
```

輸出內容會是原始字串，不會有任何遮蓋或替代類型：

```
My client's cell is 9006492568
```

用於重新識別資料的函式與去識別化函式類似：

deid.js

```
async function reidentifyWithFpe(
  callingProjectId,
  string,
  alphabet,
  surrogateType,
  keyName,
  wrappedKey
) {
  ...
}
```

這些引數會再次用於向 API 發出的要求，這次是 `reidentifyContent` 函式：

deid.js

```

// Construct deidentification request
const item = {value: string};
const request = {
  parent: dlp.projectPath(callingProjectId),
  reidentifyConfig: {
    infoTypeTransformations: {
      transformations: [
        {
          primitiveTransformation: {
            cryptoReplaceFfxFpeConfig: {
              cryptoKey: {
                kmsWrapped: {
                  wrappedKey: wrappedKey,
                  cryptoKeyName: keyName,
                },
              },
              commonAlphabet: alphabet,
              surrogateInfoType: {
                name: surrogateType,
              },
            },
          },
        },
      ],
    },
  },
  inspectConfig: {
    customInfoTypes: [
      {
        infoType: {
          name: surrogateType,
        },
        surrogateType: {},
      },
    ],
  },
  item: item,
};

try {
  // Run reidentification request
  const [response] = await dlp.reidentifyContent(request);
}

```

使用日期轉移功能將日期去識別化

在某些情況下，日期可視為機密資料，因此我們可能會想要模糊處理。**日期轉移**可讓我們隨機增加日期，同時保留日期的順序和日期之間的時間長度。一組中的每個日期都會根據該項目的專屬時間量進行位移。如要示範透過日期轉移進行去識別化，請先查看包含日期資料的 CSV 範例檔案：

[resources/dates.csv](#)

```
name,birth_date,register_date,credit_card
Ann,01/01/1980,07/21/1996,4532908762519852
James,03/06/1988,04/09/2001,4301261899725540
Dan,08/14/1945,11/15/2011,4620761856015295
Laura,11/03/1992,01/04/2017,4564981067258901
```

資料包含兩個可套用日期偏移的欄位：`birth_date` 和 `register_date`。deid.js 會接受下限值和上限值，定義選取隨機天數的範圍，藉此轉移日期：

```
node deid.js deidDateShift -c $GOOGLE_CLOUD_PROJECT resources/dates.csv
datesShifted.csv 30 90 birth_date
```

系統會產生名為 `datesShifted.csv` 的檔案，其中的日期會隨機轉移 30 到 90 天。以下是生成的輸出內容範例：

```
name,birth_date,register_date,credit_card
Ann,2/6/1980,7/21/1996,4532908762519852
James,5/18/1988,4/9/2001,4301261899725540
Dan,9/16/1945,11/15/2011,4620761856015295
Laura,12/16/1992,1/4/2017,4564981067258901
```

請注意，我們也可以指定要移動 CSV 檔案中的哪個日期欄。`birth_date` 欄位和 `register_date` 欄位維持不變。

我們來看看處理日期偏移去識別化的函式：

deid.js

```
async function deidentifyWithDateShift(
  callingProjectId,
  inputCsvFile,
  outputCsvFile,
  dateFields,
  lowerBoundDays,
  upperBoundDays,
  contextFieldId,
  wrappedKey,
  keyName
) {
  ...
}
```

請注意，這個函式可以接受包裝金鑰和金鑰名稱，類似於使用 FPE 進行去識別化，因此我們可以選擇提供加密金鑰來重新識別日期偏移。我們提供的引數會建構 `dateShiftConfig` 物件：

deid.js

```
// Construct DateShiftConfig
const dateShiftConfig = {
  lowerBoundDays: lowerBoundDays,
  upperBoundDays: upperBoundDays,
};

if (contextFieldId && keyName && wrappedKey) {
  dateShiftConfig.context = {name: contextFieldId};
  dateShiftConfig.cryptoKey = {
    kmsWrapped: {
      wrappedKey: wrappedKey,
      cryptoKeyName: keyName,
    },
  };
} else if (contextFieldId || keyName || wrappedKey) {
  throw new Error(
    'You must set either ALL or NONE of {contextFieldId, keyName, wrappedKey}!'
  );
}

// Construct deidentification request
const request = {
  parent: dlp.projectPath(callingProjectId),
  deidentifyConfig: {
    recordTransformations: {
      fieldTransformations: [
        {
          fields: dateFields,
          primitiveTransformation: {
            dateShiftConfig: dateShiftConfig,
          },
        },
      ],
    },
  },
  item: tableItem,
};
```

5. 遮蓋字串和圖片

遮蓋是另一種將機密資訊模糊化處理的方法。這項功能會將偵測到的內容替換為相符的資訊類型。 `redact.js` 示範遮蓋：

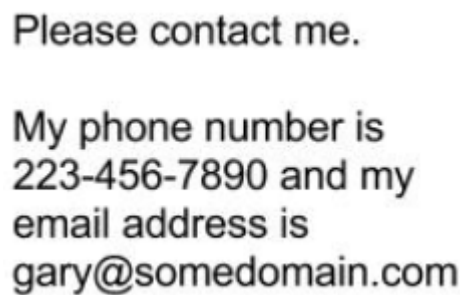
```
node redact.js -c $GOOGLE_CLOUD_PROJECT \  
string "Please refund the purchase to my credit card 4012888888881881" \  
-t 'CREDIT_CARD_NUMBER'
```

輸出內容會將範例信用卡號碼替換為資訊類型 `CREDIT_CARD_NUMBER`：

```
Please refund the purchase on my credit card [CREDIT_CARD_NUMBER]
```

如果想隱藏機密資訊，但仍需識別移除的資訊類型，這項功能就非常實用。DLP API 也能遮蓋圖片中的文字資訊。為進行示範，請參閱以下範例圖片：

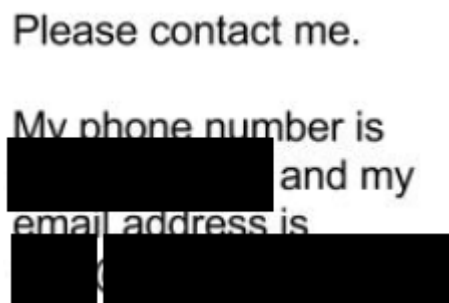
[resources/test.png](#)



如要遮蓋上圖中的電話號碼和電子郵件地址，請執行下列步驟：

```
node redact.js -c $GOOGLE_CLOUD_PROJECT \  
image resources/test.png ./redacted.png \  
-t PHONE_NUMBER -t EMAIL_ADDRESS
```

系統會根據指令將要求遮蓋的資訊塗黑，並生成一張新圖片 `redacted.png`：



以下是用於遮蓋字串的函式：

[redact.js](#)


```

async function redactText(
  callingProjectId,
  string,
  minLikelihood,
  infoTypes
) {
  ...}

```

以下是提供給 `deidentifyContent` 函式的要求：

redact.js

```

const request = {
  parent: dlp.projectPath(callingProjectId),
  item: {
    value: string,
  },
  deidentifyConfig: {
    infoTypeTransformations: {
      transformations: [replaceWithInfoTypeTransformation],
    },
  },
  inspectConfig: {
    minLikelihood: minLikelihood,
    infoTypes: infoTypes,
  },
};

```

以下則是用於遮蓋圖中資訊的函式：

redact.js

```

async function redactImage(
  callingProjectId,
  filepath,
  minLikelihood,
  infoTypes,
  outputPath
) {
  ...}

```

以下是提供給 `redactImage` 函式的要求：

redact.js

```
// Construct image redaction request
const request = {
  parent: dlp.projectPath(callingProjectId),
  byteItem: {
    type: fileTypeConstant,
    data: fileBytes,
  },
  inspectConfig: {
    minLikelihood: minLikelihood,
    infoTypes: infoTypes,
  },
  imageRedactionConfigs: imageRedactionConfigs,
};
```

步驟 6 · 約 2 分鐘

6. 清除所用資源

我們已探討如何使用 DLP API 遮蓋、去識別化及遮蓋資料中的機密資訊。現在要清除專案中建立的所有資源。

刪除專案

前往 GCP 主控台的「Cloud Resource Manager」頁面：

在專案清單中，選取我們一直在處理的專案，然後按一下「刪除」。系統會提示您輸入專案 ID。輸入後，按一下「關機」。

或者，您也可以直接透過 Cloud Shell 和 gcloud 刪除整個專案：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

步驟 7 · 約 0 分鐘

7. 恭喜！

太厲害了！你成功了！Cloud DLP 是功能強大的工具，可用來檢查和分類機密資料，並執行去識別化作業。

涵蓋內容

- 我們已瞭解如何使用 Cloud DLP API 檢查字串和檔案中的多種資訊類型
 - 我們瞭解了如何使用 DLP API 遮蓋字串，隱藏與資訊類型相符的資料
 - 我們使用 DLP API，透過加密金鑰將資料去識別化，然後重新識別
 - 我們使用 DLP API 遮蓋字串和圖片中的資料
-